RESEARCH PAPER IMPLEMENTATION

Fashion Image Generation Using Deep Convolutional Generative Adversarial Networks (DCGAN)

Implementation based on Radford, Metz and Chintala (2015)

Academic detail	Information
Student name	Gunjan P Balapure
PRN / Roll number	202401110027
Course / Subject	GEN AI
Department / Institute	AIML – MIT ACADEMY OF ENGINEERING
Academic year	2026-27

ABSTRACT

This project implements a Deep Convolutional Generative Adversarial Network (DCGAN) for generating grayscale fashion images. The implementation is based on the research paper *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks* by Alec Radford, Luke Metz and Soumith Chintala. A Generator network learns to transform 100-dimensional random noise into 28 × 28 images, while a Discriminator network learns to distinguish real Fashion-MNIST images from generated samples. The model was implemented in Python using PyTorch and trained in Google Colab using a Tesla T4 GPU. The experiment used 60,000 Fashion-MNIST training images, a batch size of 128, binary cross-entropy loss and Adam optimizers. The notebook recorded 40 training epochs in total: the 20-epoch training cell was executed twice consecutively without resetting model weights or loss-history lists. The generated samples show recognizable clothing-like structures, while some outputs remain blurry or imperfect. The project demonstrates adversarial learning and discusses training behaviour, limitations and possible improvements.

1. INTRODUCTION

Generative models learn the underlying patterns of a dataset and use those patterns to create new samples. Generative Adversarial Networks (GANs), introduced by Ian Goodfellow and colleagues in 2014, use two neural networks trained in opposition: a Generator and a Discriminator. The Generator creates synthetic samples, while the Discriminator attempts to identify whether an input is real or generated.

Deep Convolutional GAN (DCGAN) combines this adversarial framework with convolutional neural networks. Convolutional layers are suited to image data because they learn spatial features such as edges, textures and shapes. In the Generator, transposed convolutions increase spatial resolution; in the Discriminator, convolutions reduce spatial resolution while extracting features.

1.1 Problem statement

The objective is to learn an image-generation model from an unlabelled collection of clothing images and generate new grayscale fashion-like images that resemble the training distribution. The project focuses on implementing the core DCGAN methodology and observing the evolution of generated images during training.

1.2 Objectives

- Study the DCGAN research paper and its architectural guidelines.
- Prepare and normalize the Fashion-MNIST dataset.
- Implement Generator and Discriminator networks using PyTorch.
- Train the networks adversarially using binary cross-entropy and Adam.
- Visualize generated samples and training-loss behaviour.
- Identify limitations and propose improvements.

2. LITERATURE REVIEW

The selected paper, *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*, was published as an arXiv preprint in 2015 and presented at ICLR 2016. It proposed architectural guidelines for stable convolutional GAN training, including replacing pooling with strided convolutions, using batch normalization in appropriate layers, using ReLU activations in the Generator and LeakyReLU activations in the Discriminator, and using a Tanh output in the Generator.

The present project follows the central DCGAN idea but adapts the network to Fashion-MNIST, whose images are 28 × 28 grayscale pixels. The original paper's commonly illustrated image architecture uses 64 × 64 images; therefore, the architecture implemented here is an educational adaptation rather than an exact reproduction of every experiment in the paper.

3. THEORETICAL BACKGROUND AND METHODOLOGY

A GAN contains two networks with competing objectives. Let x represent a real data sample and z represent a random latent vector sampled from a prior distribution. The Generator G maps z to a synthetic sample G(z). The Discriminator D outputs a probability that an input sample is real.

The original minimax objective is:

$$\min_G \max_D V(D,G) = E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p_z}[\log(1 - D(G(z)))]$$

In practical training, the Discriminator is trained to classify real images as 1 and generated images as 0. The Generator uses the non-saturating objective: it tries to make D(G(z)) close to 1. Binary cross-entropy loss is used for both networks.

3.1 Generator architecture

The Generator receives a tensor of shape 100 × 1 × 1. A transposed convolution expands it to 256 × 7 × 7, followed by batch normalization and ReLU. Further transposed convolutions expand the feature maps to 128 × 14 × 14 and then 1 × 28 × 28. The final Tanh activation produces pixel values in the range [–1, 1].

Generator layer	Output shape	Operation
Input noise	100 × 1 × 1	Random normal vector
Transposed convolution 1	256 × 7 × 7	Kernel 7, stride 1
BatchNorm + ReLU	256 × 7 × 7	Feature normalization / activation
Transposed convolution 2	128 × 14 × 14	Kernel 4, stride 2, padding 1
BatchNorm + ReLU	128 × 14 × 14	Feature normalization / activation
Transposed convolution 3 + Tanh	1 × 28 × 28	Kernel 4, stride 2, padding 1

3.2 Discriminator architecture

The Discriminator receives a 1 × 28 × 28 image. Convolutional layers reduce the spatial dimensions from 28 × 28 to 14 × 14 and then 7 × 7. LeakyReLU is used after convolution, with batch normalization in the intermediate layer. A final 7 × 7 convolution produces one logit-like score, and Sigmoid converts it to a probability.

Discriminator layer	Output shape	Operation
Input image	1 × 28 × 28	Real or generated image
Conv2D + LeakyReLU	64 × 14 × 14	Kernel 4, stride 2, padding 1
Conv2D + BN + LeakyReLU	128 × 7 × 7	Kernel 4, stride 2, padding 1
Conv2D + Sigmoid	1 × 1 × 1	Kernel 7, stride 1

4. DATASET AND EXPERIMENTAL SETUP

4.1 Fashion-MNIST dataset

Fashion-MNIST is a benchmark dataset of 70,000 grayscale fashion-product images across 10 categories. The standard split contains 60,000 training images and 10,000 test images. Each image is 28 × 28 pixels. This implementation uses the 60,000-image training split for adversarial learning. Labels are not used by the Generator or Discriminator because the implemented DCGAN is unconditional.

The ten categories are T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag and Ankle boot.

4.2 Preprocessing

Images are converted to PyTorch tensors and normalized using mean 0.5 and standard deviation 0.5. For grayscale pixels originally in [0, 1], this transformation maps values to approximately [–1, 1], matching the Generator's Tanh output.

4.3 Software and hardware

Item	Configuration
Programming language	Python
Deep learning framework	PyTorch
Dataset utility	Torchvision
Execution platform	Google Colab
Accelerator reported	NVIDIA Tesla T4 GPU
Dataset split	60,000 training images
Image size / channels	28 × 28 / 1 grayscale channel
Latent dimension	100
Batch size	128
Configured epochs per run	20
Recorded epochs in notebook	40 total (two consecutive runs)
Loss function	Binary Cross Entropy (BCELoss)
Optimizer	Adam
Learning rate	0.0002
Adam betas	(0.5, 0.999)

4.4 Training procedure

For each batch, the Discriminator is first updated using real images and detached generated images. Its loss is the sum of real-image and fake-image binary cross-entropy losses. The Generator is then updated using a new noise batch; its generated images are assigned the real target label so that it learns to fool the Discriminator. A fixed set of 16 noise vectors is used to monitor image generation over time.

5. EXPERIMENTAL RESULTS AND ANALYSIS

5.1 Training execution

The notebook's epoch verification output reported 20 configured epochs and 40 recorded entries in each loss list. This indicates that the 20-epoch training cell was executed twice in the same active runtime without reinitializing the model parameters or clearing the loss-history lists. Therefore, the second run continued from the weights learned during the first run. The experiment is reported as 40 consecutive epochs in total, not as a single run configured with EPOCHS = 40.

5.2 Training loss behaviour

The saved loss plot shows the Generator loss decreasing from approximately 1.68 at the beginning to around 1.03 near the end of the recorded run. The Discriminator loss increases from approximately 0.70 to around 1.22. These values are visual estimates read from the supplied graph; exact values should be taken from the notebook's G_losses and D_losses arrays if required for a formal quantitative table.

The loss curves move in different directions because the networks have opposing objectives. The Generator improves at producing samples that are more difficult for the Discriminator to reject. The Discriminator's task becomes harder as generated samples become more realistic. GAN losses do not need to decrease monotonically, and loss values alone do not establish image quality or convergence.

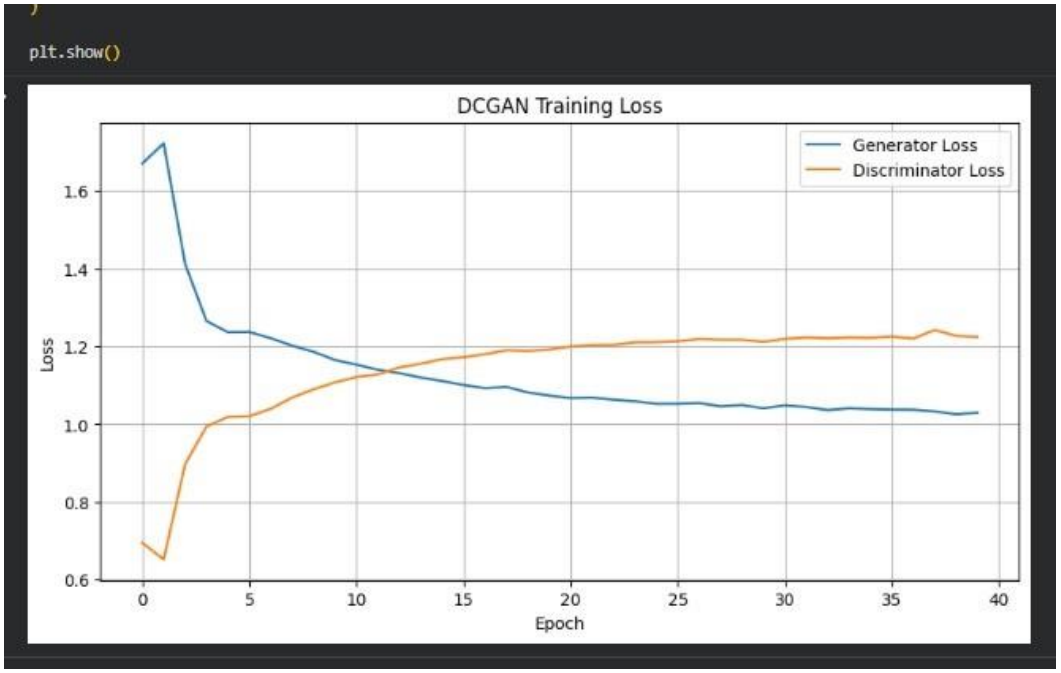


Figure 1. Training loss graph captured from the executed Google Colab notebook. The horizontal axis shows the recorded training iterations/epochs; the notebook recorded 40 loss values.

5.3 Generated image observations

The final generated grid contains recognizable clothing-like structures, including trouser-like forms, footwear, tops and bag-like shapes. The outputs demonstrate that the Generator learned some of the visual structure present in Fashion-MNIST. However, several samples have soft boundaries, uneven shapes or limited detail. This is consistent with the low 28 × 28 resolution and the relatively simple network used in this experiment.

6. GENERATED SAMPLES AND EPOCH COMPARISON

The following figures are screenshots captured from the student's Colab execution. They are included as implementation evidence rather than as externally sourced example images.

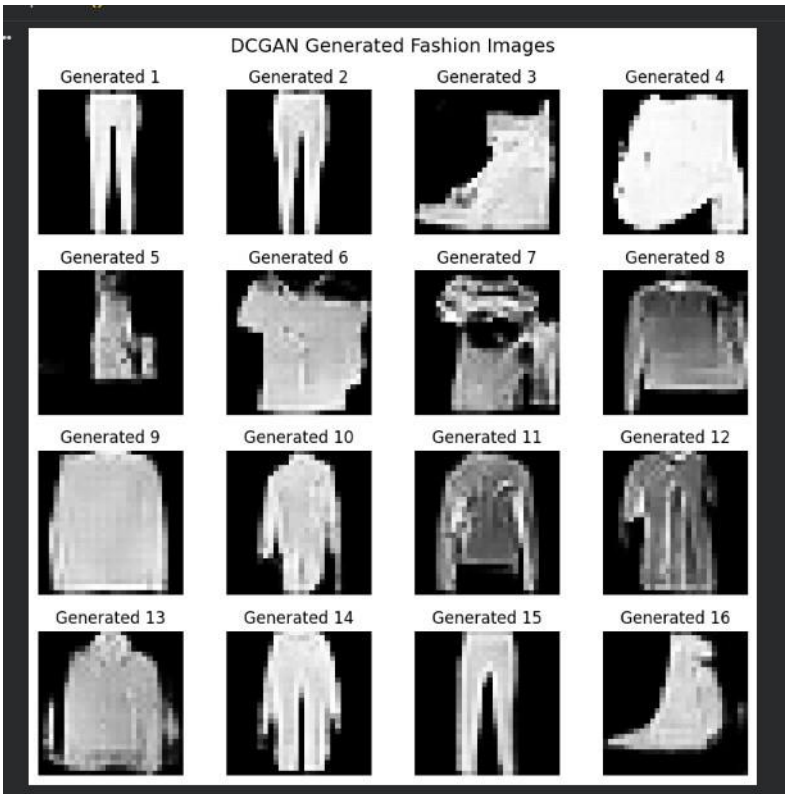


Figure 2. Final generated Fashion-MNIST samples from the trained Generator (16 fixed noise inputs).

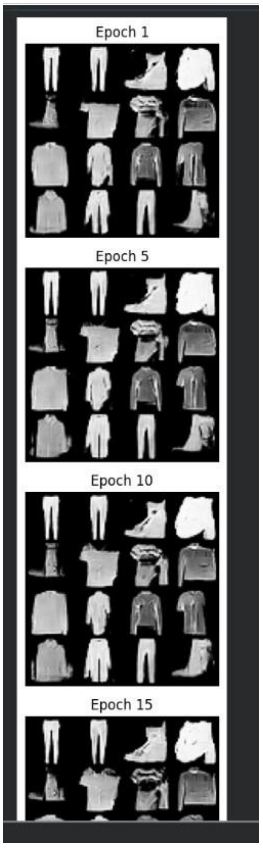


Figure 3. Saved sample grids at selected training epochs. The images are generated from the same fixed noise vectors to make visual changes easier to compare.

6.1 Interpretation

The generated images show that the model has learned broad category-level patterns, but they are not expected to reproduce exact training images. The unconditional Generator is not given a requested class label, so it does not guarantee a chosen category for each output. Visual inspection is useful for a small educational experiment, but it is subjective. A stronger evaluation would include quantitative measures such as Fréchet Inception Distance (FID), Inception Score where appropriate, and a larger blinded human evaluation. For grayscale Fashion-MNIST, feature-extractor choice and metric interpretation should be reported carefully.

7. LIMITATIONS AND POSSIBLE IMPROVEMENTS

Limitation observed / expected	Possible improvement
Blurry or coarse generated images	Use a deeper convolutional architecture or train on higher-resolution data.
GAN training can be unstable	Experiment with WGAN-GP, spectral normalization or carefully tuned learning rates.
Possible mode collapse	Monitor sample diversity; consider minibatch features or alternative objectives.
Only qualitative image evaluation was performed	Calculate FID or suitable feature-based metrics and report their limitations.
Unconditional model cannot control the output class	Implement a conditional GAN using class labels.
Small 28 × 28 grayscale dataset	Experiment with CIFAR-10 or a higher-resolution fashion dataset.

8. CONCLUSION

This project implemented the core DCGAN methodology using PyTorch and the Fashion-MNIST dataset. The Generator transformed random latent vectors into 28 × 28 grayscale images, while the Discriminator learned to distinguish real training images from generated samples. Training was performed in Google Colab using a Tesla T4 GPU, binary cross-entropy loss and Adam optimizers. The notebook recorded 40 consecutive training epochs because the 20-epoch training cell was run twice without resetting the models. The final samples contain recognizable fashion-like shapes, demonstrating that the model learned useful visual patterns. The outputs also reveal limitations in image sharpness and detail. Future work can investigate conditional generation, improved GAN objectives, larger datasets and quantitative evaluation.

9. REFERENCES

[1] A. Radford, L. Metz and S. Chintala, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks,” arXiv:1511.06434, 2015. https://arxiv.org/abs/1511.06434

[2] H. Xiao, K. Rasul and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” 2017. https://arxiv.org/abs/1708.07747

[3] I. Goodfellow et al., “Generative Adversarial Nets,” Advances in Neural Information Processing Systems, 2014. https://arxiv.org/abs/1406.2661

[4] PyTorch Documentation, “torch.nn” and “torchvision.datasets.FashionMNIST.” https://pytorch.org/docs/ and https://pytorch.org/vision/stable/datasets.html

.